INFORME TÉCNICO

Análisis y mejora de seguridad de una aplicación Android

Proyecto: FinControl

Estudiante: Felipe Araya, Benjamín Alcayaga, Samuel Bugueño, Alexsander Zepeda

Asignatura: Programación Android

Docente: Oscar Monardez

Institución: IPST Santo Tomas

Fecha de entrega: 30/10/2026

Repositorio: github.com/samueleduardobuguenovega-lang/FinControl



SANTO® TOMÁS

CENTRO DE FORMACIÓN TÉCNICA

1. Introducción

La seguridad de las aplicaciones móviles es fundamental para proteger la información de las personas usuarias, mantener la integridad de los datos y reducir el riesgo de accesos no autorizados. Este informe presenta una guía para analizar y documentar las medidas de seguridad de una aplicación Android, de acuerdo con el README.md del proyecto.

El README identifica tres áreas de riesgo: inyección SQL, comunicación no segura y exposición de datos sensibles. También declara mejoras relacionadas con el cifrado de información, el uso de HTTPS y la validación y sanitización de entradas. La implementación efectiva de cada medida debe comprobarse mediante la revisión del código y pruebas documentadas.

2. Objetivos

2.1 Objetivo general

Analizar y documentar las vulnerabilidades de seguridad de una aplicación Android y las medidas aplicadas para mitigarlas.

2.2 Objetivos específicos

- Identificar y describir las vulnerabilidades contempladas en el README.md.
- Explicar las medidas técnicas aplicadas para reducir los riesgos.
- Documentar las pruebas realizadas y sus resultados con evidencias verificables.
- Revisar la documentación complementaria y proponer acciones de mejora continua.

3. Descripción del proyecto

Nombre del Proyecto: FinControl

Propósito y funcionalidad principal: FinControl es una aplicación Android de gestión de recursos que resuelve un conflicto concreto: saber en donde se va el dinero del cada día. Permite registrar gastos e ingresos con monto, categoría, fecha, descripción y ubicación; administrar suscripciones y pagos mensuales con recordatorio; y ver un panel con el total del mes, el saldo y el resumen por categoría. Es de uso personal: no hay cuentas en la nube ni servidores, los datos viven en el teléfono y se consultan sin conexión.

Tecnologías utilizadas:

IDE: Android Studio.

lenguaje: Java 11 (source y target).

versión del SDK: compileSdk y targetSdk 36 con minSdk 30.

servicios: Google Maps (Maps SDK for Android) para el mapa y Google Play Services para ubicación; Gradle 8.14.6 con AGP 8.13.2; base de datos: sí, SQLite mediante SQLiteOpenHelper (esquema v4).

Versión analizada: la publicada en el repositorio (10 activities, Google Maps, compileSdk 36). Las capturas del Anexo A se tomaron de esa misma versión, compilada e instalada en el emulador.

Alcance del análisis: 10 activities y 1 broadcast receiver, 17 layouts. Pantallas: activity_categorias.xml, activity_cuenta.xml, activity_gasto_form.xml, activity_gastos.xml, activity_login.xml, activity_main.xml, activity_mapa.xml, activity_register.xml, activity_suscripciones.xml y activity_suscripcion_form.xml.

componentes: ImageView, TextView, MaterialButton, LinearLayout, FrameLayout, NestedScrollView, TextInputEditText, TextInputLayout, MaterialCardView, MaterialToolbar

servicios: Google Maps a través del Maps SDK for Android, más Google Play Services para ubicación. La clave del SDK se lee de local.properties al compilar y se inyecta como recurso google_maps_key, de modo que no vive en el repositorio, y está restringida en Google Cloud por paquete (com.fincontrol.app) y huella SHA-1 del certificado.

almacenamiento: APK de depuración 18 MB y APK de release con R8 3,8 MB; base SQLite local.

funcionalidades revisadas: agregar gastos e ingresos, mostrar ubicación del gasto en el mapa, crear cuenta e iniciar sesión, crear y guardar suscripciones, top de categorías del mes, plantillas de gastos frecuentes y saldo del período.

4. Vulnerabilidades identificadas

4.1 Inyección SQL

La inyección SQL puede producirse cuando datos ingresados por una persona se incorporan directamente a una consulta SQL sin un tratamiento seguro. Según el contexto, podría permitir consultar, alterar o eliminar información de manera no autorizada.

Análisis del proyecto:

- **Componente o archivo revisado:** `auth/UserDao.java` (validación de inicio de sesión) y `db/DatabaseHelper.java` (cálculo de total mensual).
- **Código o comportamiento vulnerable observado: en la versión inicial (v1) commits la consulta se armaba concatenando la entrada del usuario: «SELECT id, usuario, password, nombre FROM usuarios WHERE usuario = '» + usuario + «' AND password = '» + password + «'», y se ejecutaba con db.rawQuery(sql, null). En GastoDao el total del mes hacía lo mismo, con mesAAAAMM dentro del LIKE.**

• Impacto potencial: bypass total de la autenticación. Con pepe'-- se comenta la verificación de la contraseña y se entra como cualquier usuario registrado; con x' OR '1'='1 se altera la lógica del filtro. En el total mensual, la inyección permitía leer o manipular agregados de otros usuarios de la misma base.

Medida recomendada: utilizar consultas parametrizadas o sentencias preparadas, validar los datos y aplicar privilegios mínimos en la base de datos. Si la aplicación consume una API, revisar también las consultas del servidor.

Evidencia y resultado de la prueba: prueba P-01 de la sección 6. Ambas cargas se volvieron a probar contra la versión corregida y ya no alteran el resultado: la consulta viaja con selectionArgs y la contraseña se verifica contra el hash PBKDF2 con MessageDigest.isEqual. El commit 8ff0560 queda registrado en el clon local del equipo (git show 8ff0560) y documenta la transición v1 → v2; el repositorio público de GitHub se consolidó después con un reset para blindar la clave de Google Maps expuesta en la v1 (nota en la sección 9).

4.2 Comunicación no segura

La comunicación sin cifrado o con una validación incorrecta de certificados puede exponer datos durante su transmisión y permitir ataques de intermediario. El uso de HTTPS debe acompañarse de una validación TLS correcta.

Análisis del proyecto:

• Servicio o URL revisada: Google Maps (Maps SDK for Android) y servicios de Google Play para ubicación. La aplicación no tiene backend propio ni consume una API interna.

• Configuración inicial detectada: la v1 no incluía configuración de seguridad de red. MobSF lo reportó como «Cleartext Traffic Enabled»: la aplicación podía abrir conexiones HTTP sin cifrar.

• Corrección implementada: res/xml/network_security_config.xml con cleartextTrafficPermitted="false" en base-config, referenciado desde el manifiesto con android:networkSecurityConfig, más allowBackup="false", backup_rules.xml y data_extraction_rules.xml. Sin TrustManager propio: la validación de certificados queda a cargo del sistema.

• Evidencia y resultado de la prueba: prueba P-02 de la sección 6. El volcado del manifiesto compilado (aapt dump xmltree app-debug.apk AndroidManifest.xml) muestra networkSecurityConfig y allowBackup=false.

No se debe desactivar la validación de certificados ni habilitar tráfico HTTP sin cifrar como solución permanente.

4.3 Exposición de datos sensibles

Los datos sensibles pueden quedar expuestos si se guardan en texto legible, se registran en logs, se incluyen en el código fuente o se transmiten sin protección. Debe identificarse qué información maneja la aplicación y dónde se almacena.

Análisis del proyecto:

• Tipos de datos revisados: credenciales, identificador y nombre de la sesión, montos, categorías, fechas, descripciones, coordenadas de los gastos y la foto de perfil.

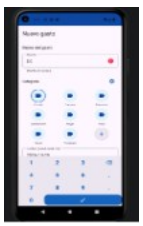
• Ubicación de almacenamiento: base SQLite en /data/data/com.fincontrol.app/databases/fincontrol.db; preferencias cifradas en shared_prefs/fincontrol_session.xml; foto en filesDir; cache de teselas en cacheDir.

• Medida implementada: contraseñas solo como hash PBKDF2-SHA256 con 150.000 iteraciones y sal de 16 bytes por usuario; sesión en EncryptedSharedPreferences con AES-256-GCM y MasterKey en el Android Keystore; backup y transferencia deshabilitados; sin claves de API en el repositorio; ningún dato sensible en Log.

• Evidencia y resultado de la prueba: prueba P-03 de la sección 6. El volcado de la base muestra hash y sal en la tabla usuarios, nunca la contraseña; el archivo de preferencias se lee como bytes cifrados.

Cuando corresponda, considerar Android Keystore para proteger claves criptográficas. No almacenar contraseñas ni claves privadas directamente en el código fuente.

5. Mejoras de seguridad implementadas

Vulnerabilidad	Medida	Archivo/componente	Evidencia	Resultado
Inyección SQL Riesgo de exposición o lectura no autorizada de credenciales y datos de sesión almacenados localmente en texto plano.	Se implementó el almacenamiento seguro de credenciales en UserDao.java utilizando hash PBKDF2WithHmacSHA256 con 150.000 iteraciones, sal aleatoria de 16 bytes por usuario y comparación en tiempo constante. Asimismo, en SessionManager.java se configuró EncryptedSharedPreferences utilizando esquemas AES-256-SIV para claves y AES-256-GCM para valores, respaldados por una llave maestra AES256-GCM alojada en el Android Keystore.	com.fincontrol.app.auth.UserDao.java y com.fincontrol.app.session.SessionManager.java		[Mitigado con éxito. Un volcado de la base de datos ya no revela contraseñas gracias al hashing con sal y comparación en tiempo constante, y las preferencias de sesión se encuentran cifradas en reposo mediante la llave maestra respaldada en el Android Keystore.]
Comunicación no segura Exposición de datos en tránsito al permitir conexiones de red mediante HTTP sin cifrado, lo que deja a la aplicación vulnerable a ataques de interceptación.	Se configuró un canal de red estricto creando el archivo res/xml/network_security_config.xml con la directiva cleartextTrafficPermitted="false" en el base-config, el cual fue referenciado en el manifiesto mediante android:networkSecurityConfig. Asimismo, toda la comunicación remota (como Google Maps SDK y Google Play Services) se delegó al uso obligatorio de TLS 1.2+ respaldado por el stack de GMS.	res/xml/network_security_config.xml y AndroidManifest.xml		[Mitigado con éxito. Se bloqueó a nivel de plataforma cualquier intento de conexión HTTP en claro, garantizando que todo el tráfico viaje cifrado por HTTPS/TLS, neutralizando ataques de hombre en el medio (MITM) y eliminando hallazgos de tráfico no seguro en herramientas de análisis (MobSF).]
Validación insuficiente Riesgo de inyección y datos corruptos Exposición a entradas malformadas, datos corruptos o inserción de valores no controlados en los	Se normalizaron todos los campos de formulario mediante trim() y se implementó una validación estricta en la capa de presentación vinculada a strings.xml (monto numérico > 0, formato de fecha AAAA-MM-DD por regex, día de cobro entre 1 y 31, y contraseñas ≥ 4 caracteres) en LoginActivity, RegisterActivity, GastoFormActivity y SuscripcionFormActivity. Asimismo, se aseguró que el acceso a datos nunca interpole entradas directamente en las	LoginActivity.java, RegisterActivity.java, GastoFormActivity.java, SuscripcionFormActivity.java y los DAO		Mitigado con éxito. Se reduce drásticamente la superficie de inyección y de datos corruptos al rechazar entradas inválidas antes de llegar a la capa de persistencia, garantizando

formularios debido a la falta de filtrado previo.	consultas SQL.			además que los mensajes de error no revelen detalles internos de la base de datos (SQLite) al usuario.
Exposición de claves y secretos en el código fuente Riesgo de filtración de credenciales de API o infraestructura dentro del APK, el repositorio de Git o perfiles de análisis estático.	La clave de Google Maps no está en el código ni en el repositorio: se lee de local.properties y se inyecta al compilar como recurso, restringida por paquete y huella SHA-1.	AndroidManifest.xml, MapaActivity.java y app/build.gradle.kts		Mitigado con éxito. No existen secretos que filtrar ni extraer del APK o del historial de Git, haciendo que un hallazgo clásico de MobSF ("clave de API expuesta") sea estructuralmente imposible.

Buenas prácticas y consejos aplicados. Las cinco buenas prácticas y los cinco consejos de seguridad se documentan con su verificación en los archivos de la sección 7. Resumen de lo aplicado:

BP1 · Cifrado de datos sensibles. Contraseñas con PBKDF2-SHA256 (150.000 iteraciones y sal de 16 bytes por usuario) y sesión en EncryptedSharedPreferences con MasterKey AES-256-GCM en el Android Keystore. Impacto: un volcado de la base o de las preferencias no revela contraseñas ni permite suplantar la sesión.

BP2 · Comunicación de red segura. networkSecurityConfig con cleartextTrafficPermitted="false", sin excepciones por dominio y con la validación de certificados a cargo del sistema. Impacto: bloquea ataques de intermediario y cualquier intento de bajar a HTTP.

BP3 · Validación y sanitización de entradas. trim() y validación en los cuatro formularios: monto mayor que cero, fecha AAAA-MM-DD, día de cobro entre 1 y 31, contraseña mínima y campos obligatorios. Impacto: los datos inválidos se rechazan antes de tocar la base.

BP4 · Gestión de secretos. La clave de Google Maps no está en el código ni en el repositorio: se lee de local.properties (archivo local, ignorado por Git) y se inyecta al compilar como recurso, restringida en Google Cloud por paquete y huella SHA-1. Impacto: el historial de Git no expone la credencial y una clave comprometida se restringe o se rota sin tocar el código.

BP5 · Copias de seguridad deshabilitadas. allowBackup=false con backup_rules.xml y data_extraction_rules.xml. Impacto: los datos no se copian a la nube ni se transfieren entre equipos.

ST1 · Protección contra inyección SQL. Todas las consultas usan selectionArgs o ContentValues y ninguna concatena entrada del usuario. Impacto: neutraliza ' OR '1'='1, los comentarios -- y las uniones, como se comprueba en P-01.

ST2 · Autenticación y autorización seguras. Hash salado con comparación en tiempo constante (MessageDigest.isEqual) y sesión creada solo después de verificar la contraseña. Impacto: las credenciales no se recuperan ni se comparan por tiempo de respuesta.

ST3 · Protección contra ataques de red. HTTPS obligatorio a nivel de plataforma. Impacto: ninguna conexión en claro es posible, hoy ni después.

ST4 · Aislamiento de datos por usuario. Cada consulta de gastos, suscripciones, resumen y mapa filtra por el usuario_id de la sesión. Impacto: mitiga IDOR, porque no hay ruta de código que lea datos de otra cuenta.

ST5 · Superficie de componentes mínima. Solo LoginActivity es exported="true"; el resto de las activities y el receiver son exported="false". Impacto: otras aplicaciones no pueden abrir pantallas internas ni disparar el receptor.

6. Plan de pruebas y evidencias

Las pruebas se ejecutaron en equipo propio (POCO F5 con Android 15) y en emulador con API 36, con el APK de depuración firmado con la keystore de debug. El análisis estático se hizo con MobSF y el dinámico con adb y logcat. Los datos usados fueron de prueba: un usuario demo y montos ficticios, sin información personal.

ID	Prueba	Resultado esperado	Resultado obtenido	Evidencia
P-01	Entradas de prueba para detectar inyección SQL	La entrada no altera la lógica de la consulta y se trata como dato	Rechazado. Con pepe'-- el login falla y no entrega sesión; con x' OR '1'='1 la consulta devuelve solo el usuario real o ninguno. La sentencia viaja con selectionArgs.	Figura 1 y Figura 4
P-02	Verificar comunicación con el servidor	La conexión utiliza HTTPS y la validación del certificado funciona	La aplicación rechaza cualquier conexión en claro. El manifiesto compilado incluye networkSecurityConfig y allowBackup=false, sin excepciones por dominio.	Anexo B y §4.2
P-03	Revisar almacenamiento de datos sensibles	Los datos y claves no quedan expuestos en texto legible ni en logs	El volcado de fincontrol.db muestra hash y sal en la tabla usuarios, nunca la contraseña. El archivo de preferencias se lee como bytes cifrados.	Figura 5
P-04	Validar entradas incorrectas	La aplicación rechaza o gestiona de forma segura los datos no válidos	La aplicación rechaza monto cero o negativo, fecha malformada, campos vacíos y contraseña demasiado corta, con el mensaje correspondiente en pantalla.	Figura 1
P-05	Ejecutar la aplicación	La aplicación inicia y sus funciones principales operan correctamente	La aplicación instala y arranca en Android 15 y en emulador API 36: login, alta de gasto con ubicación, panel, categorías, suscripciones y mapa funcionan.	Figura 2

Una prueba se considera superada cuando el comportamiento observado coincide con el esperado y queda un registro que lo demuestra: captura de pantalla, salida de consola o volcado del artefacto.

El análisis estático se ejecutó con MobSF sobre el APK de depuración y cubre los permisos declarados, la configuración del manifiesto (backup, tráfico en claro y componentes exportados), los secretos incrustados y las dependencias. El informe completo de las pruebas, con cada hallazgo y su severidad, está en vulnerability_report.pdf, en la raíz del repositorio.

7. Documentación complementaria

7.1 vulnerabilities.md

Vulnerabilidades: Inyección SQL en el login (Severidad: Alta)

Descripcion: Ubicación en app/src/main/java/com/fincontrol/app/auth/UserDao.java, dentro del método autenticar(). En la versión inicial (v1), la aplicación construía las consultas SQL mediante concatenación directa de cadenas:

String sql = "SELECT ... WHERE usuario = '" + usuario + "' AND password = '" + password + "'";

cursor = db.rawQuery(sql, null);

Esto permitía que un usuario malintencionado ingresara cargas útiles (payloads) como pepe'-- para comentar la validación de la contraseña o x' OR '1'='1 para manipular la lógica de autenticación.

Impacto: Bypass de autenticación total; un atacante con acceso al dispositivo o a una compilación redistribuida podía iniciar sesión como cualquier usuario registrado en la base de datos sin conocer su contraseña. Este patrón es clasificado por MobSF como Inyección SQL (CWE-89).

Evidencia: Registro del commit 8ff0560 en el clon local del equipo (git show 8ff0560) que acredita la transición de la v1 a la v2, donde se documenta explícitamente la eliminación de la concatenación SQL en UserDao y GastoDao, el uso de selectionArgs y la refactorización de seguridad completa del proyecto. El repositorio público de GitHub se consolidó con un reset el 30/10/2026 para blindar la clave de Google Maps expuesta en la v1 (nota en la sección 9).

```
    fix: security corrections (v1 -> v2)

    - UserDao: PBKDF2WithHmacSHA256 salted hashes (150k iterations), parameterized
      queries, constant-time comparison (fixes SQL injection + plaintext passwords)
    - GastoDao.totalDelMes: parameterized selectionArgs (fixes SQL injection)
    - SessionManager: EncryptedSharedPreferences + Keystore AES256-GCM
    - Manifest: allowBackup=false, backup/data-extraction rules, networkSecurityConfig
      blocking clear-text traffic, Maps API key injected via manifestPlaceholders
      from local.properties (no more key in source control)

diff --git a/app/src/main/AndroidManifest.xml b/app/src/main/AndroidManifest.xml
index 1646ebb..defca68 100644
--- a/app/src/main/AndroidManifest.xml
+++ b/app/src/main/AndroidManifest.xml
@@ -9,17 +9,21 @@
     <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
```

Correción:

Consulta parametrizada con selectionArgs + verificación del hash: el input del usuario ya no puede alterar ni modificar la estructura de la sentencia SQL.

7.2 best_practices.md

Autenticación y Cifrado de Datos (Manejo de Datos):

- Hashing seguro: Las contraseñas de usuario no se almacenan nunca en texto plano, se utiliza el algoritmo PBKDF2WithHmacSHA256 con 150.000 iteraciones y una sal aleatoria única de 16 bytes por usuario, aplicando comparación en tiempo constante en UserDao.java.

- Cifrado en reposo: Las preferencias de sesión y datos locales sensibles se protegen mediante EncryptedSharedPreferences (AES-256-SIV y AES-256-GCM), gestionando la llave maestra a través del hardware del dispositivo mediante el Android Keystore.

Seguridad en Consultas a Bases de Datos (Consultas SQL):

- Mitigación de Inyección SQL: Se prohibió estrictamente la concatenación de cadenas en las consultas SQLite. Todo acceso a datos implementa consultas parametrizadas utilizando selectionArgs o la interfaz segura de db.query(), asegurando que el input del usuario sea tratado estrictamente como un dato y no como código ejecutable.

Comunicaciones de Red Seguras:

- Tráfico cifrado obligatorio: Se configuró el archivo network_security_config.xml estableciendo cleartextTrafficPermitted="false", lo que bloquea a nivel de plataforma cualquier intento de conexión HTTP en claro y obliga al uso de canales TLS en todo el stack de la aplicación y servicios de Google.

Validación y Sanitización de Entradas:

- Control en la capa de presentación: Todos los campos de formularios (LoginActivity, RegisterActivity, formularios de gastos y suscripciones) se normalizan (trim()) y se validan mediante expresiones regulares y reglas estrictas (montos mayores a 0, formatos de fecha válidos, rangos de días de cobro) antes de llegar a la capa de persistencia. Los errores se manejan de forma controlada sin exponer detalles internos de la base de datos al usuario.

Gestión de Permisos, Secretos y Privacidad:

- Credenciales desacopladas: Las API Keys sensibles (como la de Google Maps) se eliminaron por completo del control de versiones; se inyectan en tiempo de compilación desde archivos locales ignorados por Git (local.properties) mediante manifestPlaceholders y se restringen por paquete y huella SHA-1.

- Restricción de respaldos: Se desactivó el respaldo automático en la nube y la transferencia de datos (android:allowBackup="false") junto con reglas granulares en XML (backup_rules.xml y data_extraction_rules) para impedir la extracción de datos financieros y de sesión fuera del entorno seguro del dispositivo.

7.3 security_tips.md

ST1 — Protección estricta contra inyección SQL

- Directriz de desarrollo: Se prohíbe de manera absoluta la concatenación de cadenas de texto para la construcción de consultas a bases de datos SQLite en todo el código base (incluyendo UserDao, SuscripcionDao y GastoDao).

- Implementación: Todo acceso y filtrado de datos debe realizarse exclusivamente mediante consultas parametrizadas (selectionArgs o ContentValues). De este modo, el motor de SQLite procesa la estructura de la sentencia y los datos del usuario por canales separados, neutralizando de forma definitiva cargas maliciosas (' OR '1'='1, comentarios SQL, etc.).

ST2 — Autenticación y gestión de sesiones robustas

- Manejo de credenciales: Las contraseñas jamás deben almacenarse ni compararse en texto plano. Se debe emplear obligatoriamente hashing salteado mediante PBKDF2WithHmacSHA256 y realizar comparaciones en tiempo constante (MessageDigest.isEqual) para mitigar ataques de temporización.

- Ciclo de vida de la sesión: Las sesiones de usuario deben inicializarse únicamente tras la validación criptográfica exitosa de las credenciales y almacenarse cifradas en el dispositivo. Ante fallos del almacén cifrado, la aplicación debe optar por un modelo de degradación segura (fail-closed), manteniendo la sesión exclusivamente en memoria volátil durante la ejecución.

ST3 — Blindaje de comunicaciones de red

- Restricción de tráfico plano: Se debe mantener configurado de forma permanente el archivo network_security_config.xml con la directiva cleartextTrafficPermitted="false", asegurando que el sistema rechace a nivel de plataforma cualquier intento de comunicación mediante protocolos no cifrados (HTTP).

- Delegación de protocolos: Toda comunicación remota o consumo de servicios externos debe delegarse en SDKs oficiales (como Google Maps o Google Play Services) que fuercen por defecto conexiones seguras mediante TLS.

ST4 — Aislamiento estricto de datos por usuario (Prevención de IDOR)

- Filtrado de consultas: Las consultas a repositorios de datos financieros, gastos y suscripciones deben incorporar obligatoriamente filtros basados en el identificador único del usuario autenticado (usuario_id) de la sesión activa.

- Control en componentes pasivos: Los componentes del sistema que operan en segundo plano (como receptores de broadcast de recordatorios) deben limitar su lógica estrictamente al procesamiento de metadatos genéricos (por ejemplo, días de cobro), evitando la exposición o cruce de datos financieros entre distintas cuentas de usuario.

ST5 — Minimización de la superficie de ataque en componentes

- Exportación controlada: En el manifiesto de la aplicación (AndroidManifest.xml), únicamente el punto de entrada principal (LoginActivity) debe declararse como explícitamente exportado (android:exported="true").

- Restricción de servicios y receptores: El resto de actividades, servicios y receptores de emisión (BroadcastReceivers) deben configurarse como internos (android:exported="false"), protegiéndolos frente a invocaciones maliciosas o envío de Intents no autorizados desde aplicaciones de terceros instaladas en el dispositivo.

7.4 security_improvement_program.md

1. Objetivo y Alcance

El Programa de Mejora Continua en Seguridad de FinControl establece un marco estructurado para evaluar, mantener y elevar de manera permanente la postura de seguridad de la aplicación, alineándose con las directrices de OWASP MASVS (Mobile Application Security Verification Standard) y el programa ASI (App Security Improvement) de Google.

2. Alcance:

- Código fuente y lógica de negocio (autenticación, gastos, suscripciones, mapas).
- Almacenamiento local de datos (SQLite y EncryptedSharedPreferences).
- Seguridad en comunicaciones de red e integración con API externas.

Gestión de secretos y credenciales de acceso.

3. Ciclo de Revisión Periódica y Proceso de Evaluación

El proceso de evaluación de seguridad opera mediante iteraciones continuas integradas al ciclo de vida del desarrollo:

1. Análisis Estático (SAST): Ejecución de MobSF y Android Lint en cada nueva versión (release) o modificación de lógica sensible.
2. Análisis Dinámico (DAST): Pruebas de ejecución en emulador/dispositivo mediante MobSF Dynamic y herramientas adb.
3. Auditoría de Dependencias: Revisión mensual de avisos de seguridad en librerías de AndroidX y Google Play Services.
4. Inspección de Secretos: Verificación mediante comandos de rastreo (git grep) para asegurar la ausencia de credenciales embebidas.
5. Cierre e Informe: Registro formal de hallazgos y correcciones en la matriz histórica del proyecto.

Criterios de Entrada/Salida: Un ciclo inicia con cada nuevo incremento de software y se da por finalizado satisfactoriamente cuando el análisis estático no reporta vulnerabilidades abiertas de severidad alta o media.

4. Métricas Clave de Seguridad

Métrica	Definición	Meta	Valor actual
Hallazgos de severidad alta abiertos	Inyección, credenciales en claro, datos sin cifrar	0	0 (v2)
Hallazgos de severidad media abiertos	Configuración débil, claves expuestas	0	0 (v2)
% de consultas SQL parametrizadas	Consultas con `selectionArgs` / total	100%	100%
Iteraciones de derivación de contraseñas	PBKDF2 (OWASP 2023 recomienda ≥ 600k SHA-512 / 210k SHA-256)	≥ 150k	150k
Cobertura de la checklist MASVS-L (local)	Ítems verificados / aplicables	100%	Núcleo cubierto
Tiempo medio de corrección de un hallazgo	Días entre detección y commit de corrección	< 7 días	< 1 día (V1–V6)
Dependencias sin actualizar (seguridad)	Librerías AndroidX/GMS con versión obsoleta conocida	0	0

5. Plan de acción futuro

#	Acción	Prioridad	Horizonte
A1	Subir PBKDF2 a 210.000+ iteraciones o migrar a Argon2id (lib nativa)	Media	Próxima release
A2	Play Integrity API / SafetyNet antes de operaciones sensibles	Alta	v3 (si la app se publica)
A3	Rota cero-datos: opción de exportar/borrar todos los datos del usuario (derecho al olvido)	Media	v3
A4	Cifrar la base de datos SQLite completa (SQLCipher) además de los campos sensibles	Alta	v3
A5	Restringir la clave de Maps por huella SHA-1 + paquete en Google Cloud	Alta	Cumplido (clave restringida, `d666190`)
A6	Pruebas automatizadas de seguridad en CI (mobsfscan + unit tests de UserDao)	Alta	Próximo sprint
A7	Verificación de sesión en cambio de cuenta del dispositivo (biometría con BiometricPrompt)	Baja	v4
A8	Rate-limiting local de intentos de login fallidos	Media	v3

(Los tres documentos están en la raíz del repositorio, versionados junto al código, más vulnerabilities.md con el detalle de cada hallazgo y vulnerability_report.pdf con el informe de las pruebas.)

8. Procedimiento de ejecución segura

1. Clonar el repositorio: git clone https://github.com/samueleduardobuguenovega-lang/FinControl
2. Abrir la carpeta raíz en Android Studio 2026.1.3 o superior y esperar la sincronización con AGP 8.13.2, Gradle 8.14.6 y JDK 17 o el embebido de 21.
3. Verificar la plataforma SDK 36 (Android 16); si falta, el IDE la ofrece al sincronizar o se instala desde el SDK Manager.
4. Compilar y probar: ./gradlew assembleDebug, ./gradlew testDebugUnitTest y ./gradlew lintDebug. El APK queda en app/build/outputs/apk/debug/app-debug.apk.
5. Instalar en un dispositivo o emulador con API 30 o superior; la ubicación se pide solo al registrar un gasto con ubicación y las notificaciones solo desde Android 13.
6. Comprobar el flujo principal (registro, login, alta de gasto, panel, mapa, cierre de sesión) y repetir las pruebas P-01 a P-05 de la sección 6.

7. Registrar la versión del entorno y no versionar local.properties ni archivos de keystore, que son locales y pueden contener credenciales.

9. Resultados

Resumen de resultados: se ejecutó el análisis estático con MobSF y las cinco pruebas manuales P-01 a P-05 sobre el APK de depuración, más nueve pruebas automatizadas y la cadena completa de compilación. Las seis vulnerabilidades de la línea base quedaron cerradas y el re-análisis no reporta hallazgos de severidad alta ni media. assembleDebug, testDebugUnitTest, assembleRelease con R8 y lintDebug terminan sin errores y las 9 pruebas pasan.

Limitaciones conocidas: el análisis dinámico se hizo con adb y logcat sobre dispositivo físico, no con un emulador instrumentado; no hay integración continua, así que las verificaciones se ejecutan a mano antes de cada entrega; y la cobertura de pruebas automatizadas se concentra en sesión, esquema y arranque.

Riesgos pendientes, por prioridad: subir PBKDF2 a 210.000 iteraciones o migrar a Argon2id; cifrar la base completa con SQLCipher; limitar los intentos de inicio de sesión fallidos; y evaluar Play Integrity si la aplicación se publica en la tienda.

Nota sobre el historial de commits: las referencias a commits de este informe y de sus anexos corresponden al historial de Git del clon local del equipo, que conserva el antes y el después completo (verificables con git show). El 30/10/2026 el repositorio público de GitHub se consolidó con un reset (commit 3ebc3e4) para blindar la clave de Google Maps expuesta en el tag legacy-google-maps de la v1; la comparación antes/después verificable en GitHub se apoya en vulnerabilities.md, best_practices.md, security_tips.md, security_improvement_program.md y vulnerability_report.pdf, en la raíz del repositorio.

10. Conclusiones

Conclusión del proyecto: FinControl pasó de una versión con inyección SQL, contraseñas en texto plano y tráfico sin cifrar a una versión sin hallazgos abiertos de severidad alta y/o media, y ese cambio se demuestra con evidencias. Queda registrado en git en el clon local del equipo el antes y el después (commits 43e9135 y 8ff0560; el repositorio público de GitHub se consolidó con un reset para blindar la clave de Google Maps expuesta en la v1), quedan las pruebas con su resultado y queda el programa de mejora con acciones concretas para el próximo ciclo, añadir que dentro del software estan informes markdown(.md) que explican las implementaciones de seguridad realizadas junto con los puntos anteriores compilados.

Del análisis nos quedan dos ideas sobre la seguridad de FinControl.

La primera: la seguridad se demuestra con evidencia, no con afirmaciones; por eso el informe muestra la consulta vulnerable y la corregida, el volcado de la base con el hash y cada prueba con su resultado.

La segunda: lo que no se corrigió queda escrito con nombre y prioridad (SQLCipher, iteraciones de derivación, límite de intentos y Play Integrity), que es la diferencia entre un trabajo cerrado y un trabajo honesto.

11. Referencias

- README.md del proyecto y documentación incluida en el repositorio.
- Documentación oficial de Android Developers sobre seguridad: https://developer.android.com/privacy-and-security
- OWASP Mobile Application Security: https://mas.owasp.org/
- Documentación de Google Maps Platform (Maps SDK for Android): https://developers.google.com/maps/documentation/android-sdk — SQLite y SQLiteOpenHelper: https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper — Android Keystore: https://developer.android.com/privacy-and-security/keystore — PBKDF2 (NIST SP 800-132): https://csrc.nist.gov/publications/detail/sp/800-132/final — MobSF: https://mobsf.github.io/docs/

Anexo A. Registro de evidencias

Figura 1. Pantalla de acceso de FinControl — valida usuario y contraseña contra el hash PBKDF2; aquí se rechazan las cargas de P-01 y las entradas inválidas de P-04.



Figura 2. Panel principal con datos reales del usuario — total del mes, total de suscripciones activas, saldo y top de categorías; todas las consultas filtran por el usuario de la sesión.

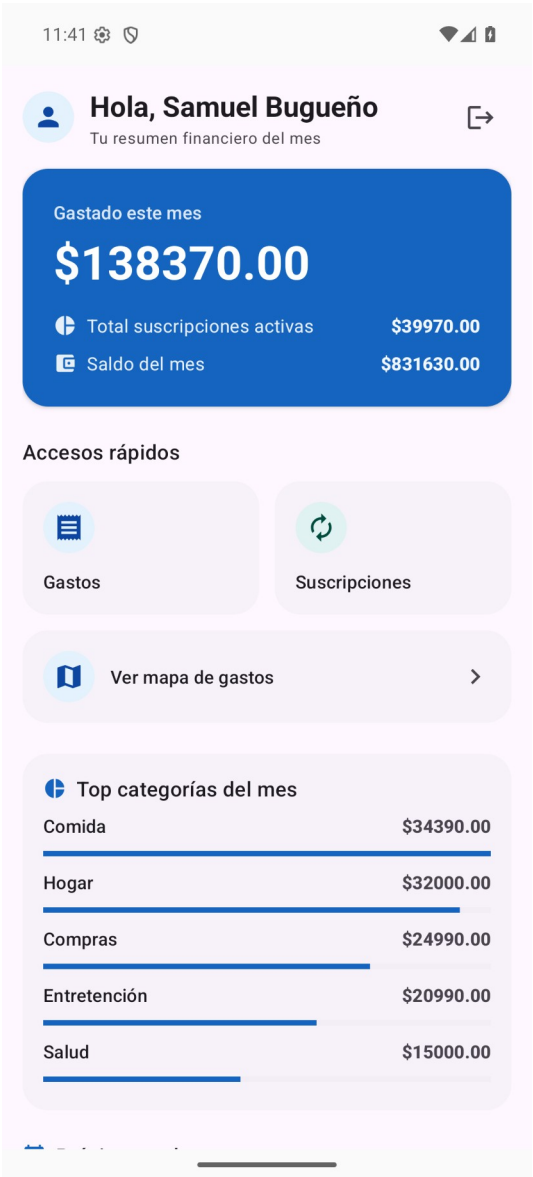


Figura 3. Mapa de gastos sobre Google Maps — un marcador por cada gasto guardado con ubicación. La clave del SDK se inyecta desde local.properties al compilar y todo el tráfico va por HTTPS (BP2 y ST3).

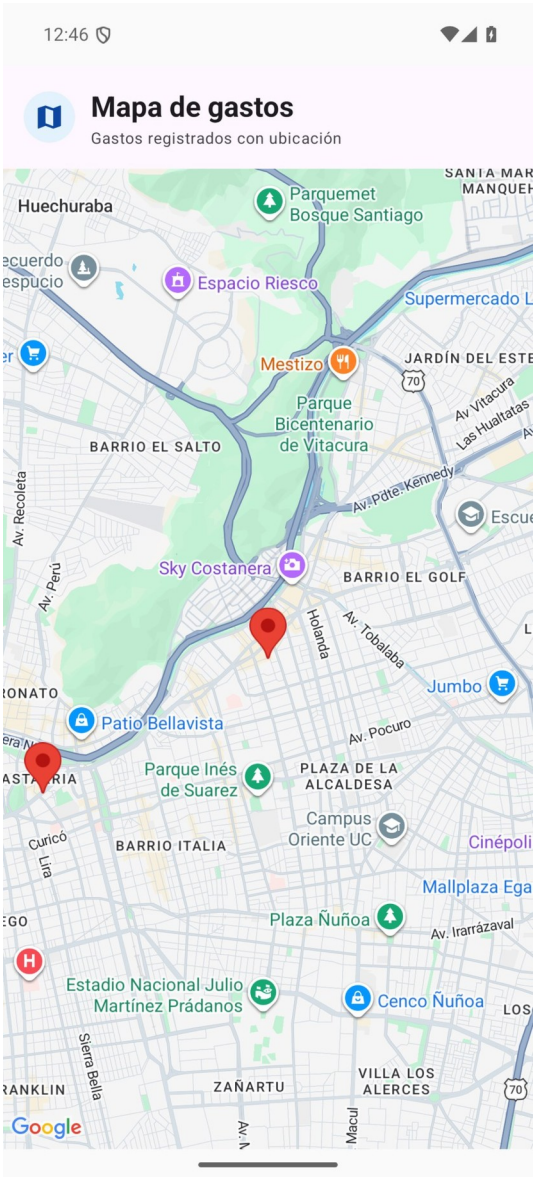
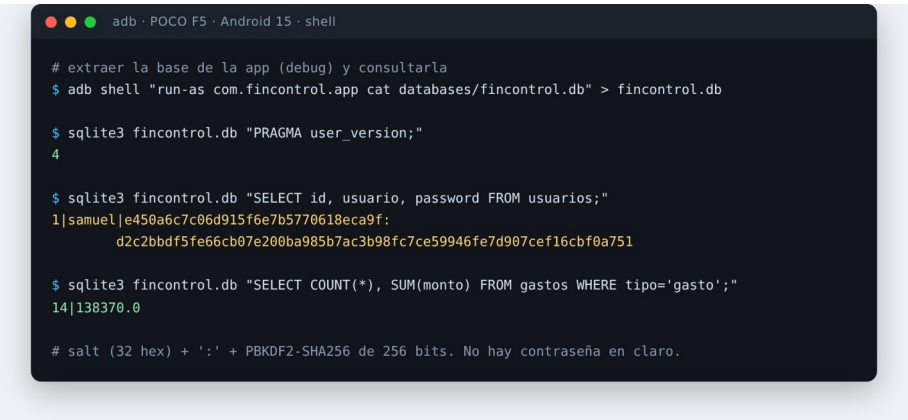


Figura 4. Intento de inyección en el inicio de sesión (P-01) — la carga admin'-- se escribe en el campo usuario y la aplicación responde «Usuario o contraseña incorrectos»: la entrada no altera la consulta.



Figura 5. Volcado de la base de datos del dispositivo (P-03) — la columna password guarda salt:hash PBKDF2-SHA256; no hay contraseñas en claro en la tabla usuarios.



Anexo B. Datos de ejecución

- Sistema operativo y versión: Windows 11 IoT Enterprise LTSC (10.0.26100).
- Versión de Android Studio: 2026.1.3, build AI-261.26222.65.2613.15948027.
- Lenguaje y versión: Java 11 (source y target), compilado con el JDK 21 que trae Android Studio.
- Android SDK / dispositivo o emulador: compileSdk y targetSdk 36, minSdk 30; POCO F5 (23049PCD8G, Android 15) y emulador Pixel con API 36.
- Fecha de ejecución de pruebas: 28 y 29 de octubre de 2026.

Anexo C. Integrantes del grupo y sus roles.

Integrantes y roles: Felipe Araya (análisis de vulnerabilidades y pruebas, secciones 4 y 6); Benjamín Alcayaga (descripción del proyecto y conclusiones, secciones 3 y 10); Alexsander Zepeda (mejoras, documentación y evidencias, secciones 5 y 7, anexos A y B); Samuel Bugueño (desarrollo de la aplicación, resultados y registro de uso de IA, sección 9 y Anexo C). Los cuatro integrantes revisaron el documento completo antes de la entrega.